

BotCortex: Sim-to-Real and Cross-Embodiment Research

OpenHorizon Labs · Research dossier · 12 September 2026

Selected direction: cross-embodiment transfer of repair knowledge, anchored on OpenArm and the Waveshare RoArm-M2-Pro and screened across simulated arms.

Working paper title: *Failures Travel: Cross-Embodiment Transfer of Repair Knowledge for Agent-Authored Robot Skills*.

Status: literature-informed research proposal and implementation plan. Existing BotCortex simulation results are identified separately from proposed experiments. No cross-robot hardware result, transfer result, or novel-method result is claimed.

Revision review — 12 September 2026: incorporates the selected RoArm-M2-Pro and owned RealSense; corrects joint-count and command-unit assumptions; credits completed runtime/web foundations; and distinguishes camera installation from validated object tracking. The review compared the PDF with web commit [37b5452](#) and runtime commit [1f92197](#), and used Aside search to verify the hardware documentation in sources [28–29](#).

Simulation pilot — 15 September 2026: the thin slice (§15) ran end to end in simulation on four bodies. Cross-body repair memory cut attempts-to-recovery from 4.61 to 2.00 on a held-out body and beat hand-written rules (2.57); negative transfer was measured (5 of 28 cases), and same-body memory about other faults was harmful (6.61). Details in §2 under *Simulation pilot* and in `botcortex-runtime/docs/research/slice_pilot.md`. Not a hardware result.

Thesis revision — 12 September 2026 (Sai's call): the primary question moves from “does active repair selection beat fixed strategies on two arms” to “does repair knowledge learned on one robot body make recovery cheaper on another.” Active selection remains the mechanism; transfer is the headline claim, an open cross-embodiment failure benchmark is the artifact. Sources [30–37](#) were added from a targeted discovery pass and are abstract-level unless stated otherwise.

1. Executive decision

Build a system in which **a failure diagnosed and repaired on one robot makes the next robot recover faster**, and measure that transfer on physical hardware. Use BotCortex's agent-authored programs, local execution, and customer-owned failure records as the experimental platform.

The first paper should answer one question:

When a taught task fails on a robot body the repair memory has never seen, does memory of diagnosed failures and verified repairs from *other* bodies reduce the real-world attempts needed to recover, compared with the same selector using no memory or memory from the target body

only?

The mechanism that makes transfer plausible is the representation: failures are recorded as **contract violations over object-centric task intent** (what should have happened to the objects), with the chosen intervention and its verified outcome, and never as joint-space traces alone. If that representation is embodiment-agnostic, failure knowledge should travel across arms with different kinematics, actuation, and sensing. If it is not, the experiment says so.

The physical anchors are **OpenArm plus the Waveshare RoArm-M2-Pro**, with an **Intel RealSense depth camera** already owned and awaiting mounting and calibration. The camera is the planned RGB-D input to an object-pose pipeline, not an already-working pose service. [29](#) Breadth comes from **simulated embodiments**: the vendored OpenArm model, a RoArm model, and several public arm models from MuJoCo Menagerie, used to screen the transfer hypothesis under a leave-one-embodiment-out protocol before the two physical comparisons. [36](#)

The RoArm-M2-Pro's advertised **4 DoF includes the end-effector actuator**: in its default clamp configuration, BASE, SHOULDER, and ELBOW position the tool, while HAND opens/closes the gripper. Compare this with **seven positioning joints plus a gripper per OpenArm arm**, rather than an unqualified “7 vs 4 joints.” The RoArm uses serial-bus servos with a JSON host interface; OpenArm uses CAN. Reach, payload, control rate, stiffness, and repeatability remain quantities to characterize during bring-up. [28](#)

The intervention set is unchanged: repair the program/planner; repair perception or frame calibration; identify controller/physical parameters; learn a bounded residual; or abstain and ask for assistance. The active selector from the previous revision (contracts, counterfactual replay, targeted measurements) is the **mechanism** under test; the **claim** is about what memory from other bodies adds to it.

Why this direction

- **Nobody has it.** The 2026 cross-embodiment literature transfers policies, representations, and demonstrations across bodies. No located work transfers *diagnosis and repair knowledge* across bodies, and no failure benchmark spans multiple physical embodiments or labels which repair actually recovered the task. [30](#), [31](#), [35](#)
- **It is the company's thesis made testable.** BotCortex's endgame is fleet failure-learning: every robot's failure makes every other robot smarter. A positive result is scientific backing for the moat; a negative result tells the company early that memory must stay per-embodiment.
- It fits the architecture: the agent authors and repairs; local executors run the task; customer-owned memory records what happened, now conditioned on embodiment and version.
- It admits a bounded experiment: two physical arms, a handful of simulated arms, a few task families, no new foundation model, no dexterous pretraining.
- It has strong nearby research to build on and to be bounded by. Fail2Progress already reconstructs failures in simulation; COMPASS already diagnoses simulator parameters; ASID already chooses identification actions; the software-agent community already attributes agent failures by counterfactual replay. [3](#), [6](#), [8](#), [10](#), [32–34](#)

Deliverables and decision gates

Deliverable	Evidence required
Portable execution substrate	Two physical adapters plus N simulated embodiments pass the same conformance suite, with explicit capability differences.
Open cross-embodiment failure benchmark	Versioned tasks and faults across all embodiments; each episode labelled with cause (where controlled), chosen intervention, and independently verified outcome; frozen leave-one-embodiment-out splits.
Transfer result	Attempts-to-recovery on a held-out body with cross-body memory vs. target-only memory vs. no memory, under matched budgets, on both physical arms.
Active repair selector	Still required as the mechanism; its standalone advantage over fixed strategies is reported as a secondary result.
Reproducible paper artifact	Programs, configurations, traces, split definitions, analysis, and representative failures, including negative transfer cases.

If cross-body memory does not help, or helps only in simulation, report that result with the benchmark. The benchmark and the SDK remain useful without a positive transfer claim.

2. Evidence scope and current BotCortex baseline

Research scope

The discovery pass screened **85 distinct URLs, including 31 arXiv links**. It combined search, primary paper pages, methods/results sections and selected appendices, repository inspection, and official conference pages. This is a focused review, not an exhaustive survey or a claim that every discovered page was read cover-to-cover.

The bibliography distinguishes core technical sources from background discovery. Paper versions are pinned where recorded. A source's reported experiment is not a reproduced BotCortex result. Search coverage alone cannot establish novelty; citation tracing and an updated review are required before submission.

The 85-URL count describes the original literature discovery pass. This revision adds targeted hardware documentation checks through Aside; it does not claim a new exhaustive literature search or a fresh replication of the cited experiments.

Implemented and previously verified

BotCortex runtime **0.0.2**, source commit `1f921978fab2c34962dba4daf55cc7b7e4d00eef`, fixes a concrete manipulation failure in native MuJoCo and browser WASM. The captured request was “Move the blue block next to the red one.” Two underlying defects mattered:

1. Geometry queries could restore an asymmetric two-finger grasp incorrectly, changing physical state merely by asking an IK or tip-position question.

2. Transit validation could accept accurate endpoint arrival while sweeping another block away.

The fix makes geometry queries preserve full simulation state and adds a deterministic object-motion guard. Candidate routes reject lost grasps and unintended displacement of unheld objects above 15 mm; alternate routes are tried before refusal. This is documented in

`docs/SIMULATION_FAILURE_FIX.md`.

Evidence	Recorded result and scope
Captured simulated transfer	Blue moved from <code>[0.32, -0.05, 0.222]</code> m to <code>[0.2248, -0.2246, 0.22]</code> m.
Bystander preservation	Red and green retained horizontal position; 2 mm vertical movement was initial settling.
Backend coverage	Native MuJoCo and the shipped browser WASM wheel passed the reproduced task and tested variants.
Earlier verification	228 runtime tests; 164 web tests; typecheck/build; 20 fixture smoke checks; physical simulation smoke.
Observed latency	About 59–70 seconds initial computation, followed by roughly 30 seconds paced playback. These are separate quantities.
Hardware transfer	Not established by these tests. No new hardware evaluation is reported here.

These checks were run during the implementation work, not rerun to generate this document. The current result is a useful regression case, not a benchmark-wide success rate.

Completed foundations to build on

The research starts from implemented components, not an empty runtime. The source review and the resolution records in `docs/CLAUDE_REVIEW.md` and `docs/SIMULATION_FAILURE_FIX.md` support the following status:

Foundation already present	Remaining research extension
Native/WASM rehearsal saves and restores physics state; geometry queries preserve the grasp.	Export versioned replay packets, reconstruct hardware state, and evaluate competing parameter hypotheses.
Route checks reject lost grasps and displaced bystanders; failed rehearsals identify the failing step.	Generalize to task-specific allowed effects, unknown observations, and independent task-success predicates.
Motion/object logs and before/after scene evidence exist. Saving edited skill code clears its prior run marker.	Record synchronized hardware observations, actual actions, overrides, and separate task/adaptor/calibration hashes.
Browser skills/episodes persist in account-scoped storage; account transitions and cancelled startup clean up old workers/leases.	Add embodiment/version-aware retrieval and enforce research train/validation/test isolation.

Foundation already present	Remaining research extension
Browser run routing, retry ownership, request deadlines, connection health, and shipped-wheel schema/provenance checks were fixed.	Complete runtime event sequencing/reconnect evidence and durable research episode export.

The workspace loading skeleton is also implemented. The earlier R1–R9 web review findings are resolved; they should not be carried forward as open research tickets. The transcript outbox is still in-memory, however, and account isolation is not the same as experiment-split isolation.

Outcome semantics matter: `RobotSession._ran(..., ok=True)` and the skill `.ran` marker mean execution completed without a reported refusal/crash. They do not independently certify the requested task outcome. The current runtime supplies scene evidence for interpretation; a task-contract verifier remains research work.

Simulation pilot: the thin slice, run (15 September 2026)

The milestone in §15 was built and run in the runtime repository (`botcortex/research/`, `docs/research/slice_pilot.md`). Four simulated bodies — OpenArm v1, a primitive RoArm-M2 model, and the Menagerie Panda and xArm7 — run one unchanged pick-and-place program through one descriptor-driven runtime. Eight controlled faults (four program, four perception) each have one inverse repair; an independent verifier grades the task from simulator state the program never reads; every (body, fault, repair) trial is simulated once into a frozen outcome table, and every selector is scored against that table. Retrieval for the target body excludes the target itself; the target-only arm is leave-one-fault-out.

Arm	Mean repairs to verified recovery (28 cases, budget 8)	Saved vs no memory	Worse than no memory	First pick in the right family
No memory / fixed strategy	4.61	0	0/28	14/28
Hand-written rules, no memory	2.57	+2.04	1/28	26/28
Target-body memory, other faults only	6.61	−2.00	20/28	16/28
Cross-body memory, contract signatures	2.00	+2.61	5/28	26/28
Cross-body memory, trace text (H2 control)	2.32	+2.29	2/28	21/28

What the pilot supports, and only in simulation: repair knowledge from other bodies reduces recovery attempts on an unseen body (H1 direction), beating the rule baseline; transfer is perfect where a fault leaves the same contract signature on every body (identity swap, occlusion, wrong object: one attempt) and degrades to negative where signatures diverge (pose bias and stale pose: three distinct signatures across four bodies). Same-body memory about *other* faults harms recovery under this

retrieval rule. The contract-versus-trace gap (H2) is small — 2.00 against 2.32 — because error text already carries object-level information; a sharper H2 test needs faults whose traces and contract violations disagree.

Targeted measurement, same day. Step 4 of §7 was added and scored on the same frozen tables: a passive re-observation and an active bounded nudge (20 mm, then re-observe), chosen by a hand-written rule when the signature is ambiguous between a biased and a stale estimate, with a random-probe baseline. The nudge separates the two on every body where both faults bit (a biased estimate tracks it, a stale one does not); the passive probe separates nothing. Charged as a full attempt, the probe saves nothing — a two-way ambiguity the passive order already tries back to back can save at most one attempt for one spent. Charged at a quarter of an attempt it helps modestly (cross-body memory 2.00 → 1.89), on the four cases where the rule fires. The quantity that decides whether active measurement pays is therefore the cost of a probe relative to a task attempt, which the hardware study must measure; the probe's accuracy is not in question.

What it does not support: any hardware claim, any learned-selector claim, and any claim about faults beyond the eight injected. The RoArm model carries placeholder servo gains. Four of the thirty-two injections caused no failure on their body and were not scored. Per the phase 2b gate, the signal is measurable in simulation for program and perception faults, so the hardware track proceeds with H1 intact; the negative-transfer mechanism is now a stated part of the method to fix (retrieval must weight exact signature matches over partial ones, or abstain).

Portability gaps found in the source

In the inspected BotCortex package, `openarm_v1` remains the only platform directory with a descriptor, backed by mock/simulation execution. There is no RoArm adapter yet. Descriptors do not make every backend generic: `jointmap.py` fixes seven joints per arm, `kinematics.py` encodes an OpenArm-specific four-joint reach solution and grasp geometry, and `safety.py`/`scene.py` identify robot bodies by the `openarm` name prefix. Adding a descriptor directory alone will not make a RoArm or a Menagerie arm work; those modules must take joint set, reach solver, and body naming from the capability pack. `config.py` still selects platform/limits at module scope. Scene queries use privileged simulator state. A functioning `RealRobot` execution path is absent from this package, and `cli.py` still rejects `--execute` as unwired.

This package boundary is important: the existing Thor/OpenArm lab runbook separately records working LeRobot teach/replay helpers and a joint-space sim bridge. Those assets can support integration; they are not yet a BotCortex hardware adapter or evidence of object-level transfer across both selected robots. No fresh hardware validation was performed for this review.

The research therefore needs real adapter and sensing work before hardware experiments. It should not present the existing browser simulator as an already-portable deployment SDK.

3. The two anchor references

Play2Perfect: what transfers, and what it costs

Play2Perfect: What Matters in Dexterous Play Pretraining for Precise Assembly? was reviewed at arXiv [2606.26428v3](#), dated 8 September 2026. It learns a dexterous prior from task-agnostic simulated play and finetunes it for precise assembly. [1](#)

The real platform is a **7-DoF KUKA arm with a 22-DoF Sharpa hand**. Reported operating rates are **60 Hz policy inference, 30 Hz object tracking, and 120 Hz training physics**. Play pretraining takes approximately **seven days on one RTX A6000**, using **24,576 parallel environments**; downstream finetuning takes about **one day per task**, using **12,228 environments**. Faster downstream learning is meaningful, but the reusable pretraining cost must remain visible.

Reported hardware results include:

Experiment	Author-reported outcome	Interpretation
Tight insertion, 10 mm clearance	10/10	Small trial count on the tested setup.
Tight insertion, 2 mm clearance	9/10	Tighter contact constraints reduce reliability.
Tight insertion, 0.5 mm clearance	6/10	Precise assembly remains difficult despite transfer.
Beam assembly, separately evaluated steps	8/10 and 7/10	Per-step rates are not end-to-end completion.
Additional end-to-end beam assembly	12/20	Separate appendix evaluation; do not replace it with step rates.
Screw-leg insertion / full screwing	7/10 / 5/10	Insertion and complete task success differ.

The policies use object-pose estimates and externally specified task sequencing and goal poses. Occlusion and contact mismatch remain failure sources. The paper describes short-horizon assembly skills, not a complete autonomous general-purpose assembly system.

The MIT-licensed repository provides an inference/deployment core, while actual deployment still requires matching observation layouts, action scaling, object tracking, robot I/O, and controllers. Its deployment guide is not a turnkey hardware stack. The project also has an interactive browser MuJoCo demonstration; browser simulation itself is not a research contribution.

BotCortex takeaway: a pretrained dexterous policy can eventually be one local executor. Do not couple the first diagnosis paper to reproducing this expensive pretraining regime or to acquiring an equivalent dexterous hand. Reuse the ideas of structured observations, contact-sensitive evaluation, and explicit transfer assumptions.

DexGPT: reconstruction is not execution success

The reviewed repository is `Hu-xiao-max/dexgpt`, commit `03ba8a26eaf9dfd272ac1a22c0fd06ccb9b2e5b`, described as “GPT for real2sim.” It reconstructs and retargets a monocular human GIF to two Sharpa hands in MuJoCo. The review inspected the README, retargeting and rollout code, and the stored physics report. [2](#)

Its saved report contains `task_success: false`, maximum penetration **5.623 mm** against a **less-than-5 mm** criterion, peak simulated contact force **314.7 N**, and hinge RMSE **5.177 degrees**. These numbers are from that saved simulation report, not new measurements or evidence of safe hardware forces.

The reconstruction assumes scale/depth, fixes the object's base, and prescribes wrist motion in place of a complete arm controller. No hardware execution controller was found in the reviewed path. A visually plausible motion or kinematic replay therefore cannot establish manipulation success, feasible arm dynamics, or real-world transfer.

No top-level license was identified in the reviewed snapshot; repository metadata returned no license. Third-party asset licenses are separate. Treat reuse permission as unresolved before incorporating its code or assets.

BotCortex takeaway: demonstrations can initialize object relations, task hypotheses, and candidate motions. They must then pass explicit kinematic, contact, effect, and hardware checks. Preserve failed physics reports rather than promoting a rendered video to a successful trial.

4. Related work and the novelty boundary

The following matrix focuses on overlaps that can invalidate an overly broad claim. It is a positioning aid, not a ranking of papers or a substitute for reproducing their methods.

Source	Relevant prior contribution	Boundary for BotCortex
ASPIRE 3	Trace-guided program repair, reusable skills, evolutionary search, cross-embodiment examples.	Code repair and a growing skill library are already established.
CaP-X 4	Coding-agent evaluation with perception/control services and real deployment.	Generating robot code through an agent harness is not enough.
Nautilus 5	Typed policy/robot/environment contracts, isolated execution, reproducible interfaces.	Interface portability is not equivalent to unchanged-policy task success.
Fail2Progress 6	Reconstructs failures and generates targeted simulated data to improve skill-effect models.	Replay-based failure classification already has direct precedent.
Counterexample-guided abstraction repair 7	Repairs symbolic/geometric abstractions from counterexamples.	Contract violations driving repair are not intrinsically new.

Source	Relevant prior contribution	Boundary for BotCortex
COMPASS / What Went Wrong? 8	Causal identification of simulator parameters responsible for real/sim trajectory mismatch.	Physics-parameter diagnosis must be compared with existing causal calibration.
RAPT 9	Predictive out-of-distribution detection, failure localization, semantic diagnosis.	Monitoring plus a natural-language explanation is insufficient novelty.
ASID 10	Active exploration for system identification.	Choosing informative calibration actions is prior art.
Phys2Real 11	Physical priors and interactive online adaptation with uncertainty.	Uncertainty-aware physical adaptation is not an untouched area.
Neural Fidelity Calibration 12	Informative online simulator adaptation.	A better twin alone needs a specific advance over adaptation methods.
TRANSIC 13	Sim-trained base policy plus a gated residual learned from human corrections.	Residual recovery and human-in-the-loop transfer already exist.
HIL-SERL 14	Real-world learning from demonstrations, interventions, and learned rewards.	Direct hardware teaching is a viable lane, not a new contribution by itself.
RialTo 15	Real-to-sim-to-real robustification using reconstructed environments.	Building and training in a digital twin has close precedent.
CEI 16	Functional 3D representations and aligned trajectories across embodiments.	Transferable representations must be distinguished from general adapter design.
UHAS 17	A shared hand action space for cross-embodiment manipulation.	Shared actions still need embodiment-specific dynamics and controller calibration.
ReKep 18	Relational keypoint constraints and closed-loop optimization.	Object-centric constraints and geometric replanning are established ideas.
Cross-environment failure reasoning data 30	Scales failure-reasoning training data for VLA/VLM models across environments.	Failure <i>reasoning</i> across environments exists; transfer of <i>repair decisions</i> across bodies, validated on hardware, does not appear to.
LabRobFail 31	Simulated fault-injection benchmark with perception/grasp/motion/logic/safety categories for a lab robot.	A single-embodiment, simulation-only failure benchmark exists; ours must add multiple physical embodiments and repair-outcome labels.

Source	Relevant prior contribution	Boundary for BotCortex
Causal Agent Replay, AgentDebugX, Repair-or-Resample 32–34	Counterfactual replay and attribution for software LLM-agent failures.	Counterfactual attribution is established for software agents; porting it to physical deployments with real sensing cost is the extension, not the idea.
Data Analogies 35	Paired demonstrations aligned across embodiments drive cross-embodiment policy transfer.	Cross-embodiment transfer of <i>policies</i> is well studied; this work is the closest analogue for aligned data across bodies.
Visual-symbol failure diagnosis 37	Diagnose, correct, and learn from manipulation failures via visual symbols.	Symbolic failure diagnosis on one robot is prior art; embodiment-conditioned memory across bodies is the boundary.

The closest scientific collision

Fail2Progress is the closest method: simulated reconstruction distinguishes inaccurate symbolic predictions from a sim-to-real gap, and generates failure-targeted data for effect-model improvement. BotCortex cannot claim to invent “replay a failure to decide what went wrong.” [6](#)

LabRobFail is the closest benchmark: controlled fault injection with a cause taxonomy, in simulation, on one embodiment. BotCortex cannot claim to invent “inject labelled faults and benchmark diagnosis.” [31](#)

The cross-embodiment literature is the closest framing: policies, representations, and paired demonstrations transfer across bodies. BotCortex cannot claim to invent “knowledge transfers across embodiments.” [16](#), [17](#), [35](#)

The uncovered intersection is the claim: repair knowledge (diagnosis, chosen intervention, verified outcome), expressed as object-centric contract violations, transferring across robot bodies and reducing attempts-to-recovery on a body it has never seen, measured on physical hardware, with the benchmark released. Each ingredient exists; the combination and its measurement do not appear to. Citation tracing must recheck this before submission.

COMPASS and ASID still bound the mechanism: identifying causal simulator parameters and choosing informative identification motions are existing methods. The active selector is not the headline. [8](#), [10](#)

Additional context

Grounding Sim-to-Real Generalization, DrEureka, Re³Sim, and RoboInspector inform transfer evaluation, randomization, reconstructed environments, and failure feedback. They are included as supporting references, with the reading scope stated in the bibliography. No headline metric from these sources is treated as a BotCortex result. [19–22](#)

5. What “portable” and “unchanged” must mean

A task can retain its meaning across robots while requiring different joint trajectories, grasp choices, controllers, or policy weights. A robot with a different reach, gripper, force capability, or sensing setup may not be capable of the same task at all.

Transfer claim	What stays fixed	What may legitimately change
Task-semantic transfer	Goal, allowed effects, invariants, parameter meaning.	Robot-specific motion and executor selection.
Program transfer	Exact task source hash.	Adapter, calibration, planner output, declared bindings.
Policy transfer	Exact weights and observation/action contract.	Only explicitly reported preprocessing and controller bindings.
Adapter compatibility	API and conformance behavior.	Embodiment-specific driver and kinematics.
Trajectory replay	The stated trajectory representation and timing.	Retargeting changes must be disclosed; joint replay is generally robot-specific.

Record separate hashes for **task source**, **policy weights**, **adapter**, **calibration**, and **simulator/model**. “Zero-shot” must name the quantity that received no target-domain optimization and disclose all target-specific setup. Calibration-free, demonstration-free, unchanged weights, and unchanged program are different claims.

Example: blue beside red

The task intent should express blue being supported by the table, in a specified spatial relation to red, with uncertainty-aware tolerances. During carrying, require maintained grasp. Preserve objects declared as bystanders. Verify the final state after release and a settling interval.

The existing no-unheld-object-motion transit guard is appropriate for this pick-and-place case. It is not a universal manipulation contract: **intentional pushing must allow motion of the designated pushed object**. A task-specific allowed-effect set prevents a useful guard from banning the task itself.

Feasibility is a first-class result

Binding a skill to a platform should produce one of: supported; supported after specified calibration or executor adaptation; unsupported; or insufficient evidence. Do not silently replace a missing force controller with a position approximation and call the program portable.

The product promise is reusable task intent and controlled adaptation on supported robots. It is not guaranteed identical behavior on arbitrary hardware.

6. Research questions and falsifiable hypotheses

Primary hypothesis

H1 (transfer). For a target embodiment held out of the repair memory, a selector conditioned on repair memory from other embodiments reaches verified recovery in fewer target-body attempts than the same selector with (a) no memory and (b) memory from the target body only, under matched repair operators, observations, and attempt budgets. Evaluate zero-shot (no target episodes) and few-shot (k target episodes added) regimes.

H2 (representation). The transfer effect in H1 depends on failures being recorded as object-centric contract violations with intervention outcomes. Replacing that record with joint-space traces plus free-text lessons removes most of the effect.

Secondary hypotheses

H3 (mechanism). Given matched repair operators and budgets, a selector using contracts, counterfactual replay, and targeted measurements reduces real-world attempts to verified recovery compared with fixed repair strategies and trace-only selection. This was the previous primary hypothesis and remains a reported result.

Secondary questions

1. Which failure classes transfer across bodies and which do not? Program and perception faults are expected to transfer better than controller/physics faults, which are body-specific by construction.
2. Does transfer degrade with embodiment distance (joint count, actuation, gripper geometry, sensing), and can capability-pack fields predict that degradation?
3. Does active measurement add value beyond passive replay, after charging its real-world cost, and does that value change when cross-body memory is available?
4. Can uncertainty-aware abstention lower wrong-repair and false-success rates without making the system unhelpfully reluctant?
5. When does cross-body memory cause **negative transfer** (a wrong repair chosen because another body's lesson did not apply), and can version/capability conditioning suppress it?

Candidate failure classes

Class	Examples	Candidate intervention
Program / planner	Wrong frame usage in code, infeasible waypoint sequence, missing prerequisite, incorrect effect assumptions.	Edit the failing step, regenerate the plan, or repair the effect model.
Perception	Wrong object identity, occlusion, stale pose, biased tracking.	Reobserve, change viewpoint, repair perception configuration.

Class	Examples	Candidate intervention
Frame / TCP calibration	Camera-to-base or tool-center offset, unit convention mismatch.	Validate transforms and recalibrate the affected mapping.
Controller / physics	Tracking delay, gain mismatch, friction, mass, compliance, grasp/contact discrepancy.	Identify parameters, update the twin/controller binding, or use a bounded residual.
Compound / unresolved	Multiple causes or observationally equivalent explanations.	Select an informative measurement or abstain.

Infrastructure failures such as stale workers, dropped commands, or mismatched artifacts must not be mislabelled as physical dynamics. Detect them before entering the physical diagnosis model, and retain their traces for operational analysis.

Identifiability limits

A misplaced object can be explained by a bad camera transform, a wrong TCP, tracking error, a bad pose estimate, or flawed program coordinates. Replaying one trajectory in one simulator does not establish which explanation is causal. The method needs competing hypotheses and measurements that can discriminate between them.

The research claim should therefore be **effective intervention selection under measured uncertainty**, not universal causal identification. Controlled injected faults permit stronger label-based evaluation; naturally occurring failures need independent adjudication and may remain unresolved.

7. Proposed method: diagnose, measure, repair

This section specifies a design to implement and test. It is not a description of an already-trained selector.

Step 1 — Capture a replayable evidence packet

At every primitive boundary, record task and artifact versions, intended action, measured action/state, timestamps, object estimates and uncertainty, controller status, contacts when observable, contract verdicts, human overrides, and outcome evidence. Keep raw observations or referenced recordings sufficient to revisit disputed diagnoses.

Extend the existing motion/object logs, failed-step labels, and browser run bindings. The executor and verifier must agree on run identity and timebase. Missing observations are explicit missing data, not zero values. The verifier produces its result independently of the authoring agent's explanation.

Step 2 — Localize the first supported violation

Evaluate preconditions, invariants, and postconditions with measurement validity and tolerances. Examples include grasp maintained during transit, object motion consistent with the allowed-effect set, and the target supported at the intended location after release.

Distinguish “predicate false” from “cannot observe predicate.” Joint arrival alone does not prove grasp or placement. The first visible violation is a useful localization point, but is not automatically the root cause.

Step 3 — Construct competing counterfactuals

Replay the relevant primitive or short segment from the reconstructed state, varying one candidate explanation at a time where possible. Examples: correct the waypoint while holding calibration fixed; change the TCP estimate; vary tracking latency; or perturb friction within identified bounds.

Use ensembles when state reconstruction or physics parameters are uncertain. A counterfactual that explains the trace is a candidate explanation, not proof. Build on the existing native/WASM `rehearsal()` state restoration; a persistent, versioned counterfactual replay service is the extension. Geometry-query purity is already fixed in 0.0.2 and should remain a regression requirement.

Step 4 — Choose a targeted measurement

When hypotheses remain ambiguous, select a bounded diagnostic observation or action: reobserve a fiducial or object; check a known tool pose; execute a low-amplitude tracking probe; or test contact at an operator-approved configuration.

A starting decision rule can rank candidate measurements by expected hypothesis reduction relative to time and execution cost. Compare it with random probes and a simple rule-based choice. Do not label a hand-written heuristic a learned causal policy. The simulation pilot (§2) implemented exactly the hand-written version: one active probe, one passive, a rule for when to spend, and a random baseline. Its lesson is that a probe's value is bounded by the size of the ambiguity it resolves and decided by its cost relative to an attempt — measure that ratio on hardware before designing an information-gain policy around it.

Step 5 — Select the repair

Estimate each available repair's chance of recovery, hardware-attempt cost, human time, and applicability. A conceptual objective is:

```
choose repair minimizing:
  expected attempts to verified recovery
  + weighted human time
  + weighted execution cost
subject to capability and operating constraints
```

The weights and treatment of failed or censored episodes must be fixed before final evaluation. Operators unavailable on a robot are masked, not imputed as successful. An abstention is an explicit decision, counted in coverage and cost.

Step 6 — Re-execute and verify

Apply the chosen intervention as a versioned change. Rehearse when supported, then run a supervised hardware attempt within the experiment protocol. Check task effects independently. A fluent explanation, completed function call, or simulation pass is insufficient to label hardware success.

Step 7 — Store the intervention outcome

Memory must include the pre-repair evidence, hypothesis, chosen intervention, changed artifacts, and verified result, including failed repairs. Every record carries the embodiment's capability-pack identity and the artifact versions. Retrieval conditions on the *target* robot's capabilities and relevant versions, and can draw from records of other embodiments; that cross-body retrieval is the object of H1, so it must be switchable per experiment arm. Separate training, validation, and test memory; freeze or explicitly constrain online updates to prevent leakage.

8. Portable SDK and runtime architecture

Architectural rule

The LLM is outside the real-time control loop. It authors or repairs a task, while local primitives, planners, or learned policy executors run it. Teaching normally uses frontier cloud models; taught execution and local memory should not depend on a live model call. A local authoring model remains an optional configuration.

This preserves the BotCortex product direction: chat authors tasks and the runtime owns execution. A plan view supports inspection of execution, while an enforced proposal/approval/execute workflow remains open work; it is not implied by a successful physics rehearsal. The hosted web UI should not be described as an already-offline interface merely because execution can be local.

Capability packs

A robot descriptor is necessary but insufficient. A capability pack should contain:

- Kinematic tree, collision geometry, joint ordering, units, tool frames, TCP, and gripper semantics.
- Position/velocity/torque limits, available control modes, measured control rate and latency, and stop behavior.
- Sensor types, calibration, timestamp sources, observation conventions, and uncertainty estimates.
- Identified dynamics and contact assumptions where relevant, with confidence and validity range.

- Driver, firmware, model, controller, calibration, and schema versions.
- Supported primitives, policy interfaces, task affordances, and conformance-test results.

Two robots can expose the same primitive name while providing different capabilities. Bind only when the required semantics are supported; do not hide absent sensing or unsupported control modes behind a uniform function signature.

Component boundaries

Component	Responsibility	Required output
Platform adapter	Translate bounded runtime commands into robot-specific I/O.	Measured state, acknowledgements, timing, status.
Perception / frame service	Maintain object estimates and transform validity.	Timestamped estimates with uncertainty and provenance.
Skill contract layer	Declare goals, allowed effects, invariants, and capability requirements.	Machine-checkable contract and binding result.
Executor registry	Select primitive, planner, task policy, or VLA executor.	Versioned executor and declared observation/action contract.
Simulator service	Rehearse and replay controlled hypotheses.	State snapshots, traces, contract outcomes, model identity.
Evidence recorder / verifier	Record execution and independently determine outcomes.	Auditable episode and verdict.
Diagnosis / repair service	Compare hypotheses, request measurements, select intervention.	Decision, uncertainty, rationale grounded in evidence.
Local memory	Retain lessons and intervention outcomes.	Version-aware retrieval with split isolation.

Implementation direction

Integrate the existing OpenArm lab driver path into a BotCortex adapter, then implement the RoArm-M2-Pro adapter against the same conformance suite. Remove joint-count, naming, and gripper assumptions from shared planning and simulation logic. Make platform state instance-specific so two different robots can coexist in a process or evaluation harness.

For the RoArm, choose and pin one interface convention. The shared M2 control docs specify **radians for JSON T=101/102 joint positions and T=105 joint feedback**, and **millimetres for Cartesian XYZ**. The Cartesian `t` parameter is the clamp/wrist angle, not a fourth spatial coordinate. The current BotCortex primitives use degrees, so conversion belongs explicitly at this adapter boundary. Raw encoder-position tick conversion is needed only for a lower-level servo interface; it is not required for JSON angle commands. [28](#)

Angular position and motion-rate units are separate: T=101/102 `spd` and `acc` use documented step-based conventions, not rad/s and rad/s². The vendor Python SDK also remaps gripper coordinates relative to raw JSON. Validate command/feedback round trips and stopping/queue behavior against

the installed firmware, rather than mixing SDK, JSON, and raw-servo conventions or assuming a command acknowledgement means the motion completed.

Use native/headless simulation for repeated training and diagnosis experiments. Keep browser simulation for inspection, onboarding, and interactive rehearsal. Browser and native paths should share semantics and contract fixtures, but performance equality is not assumed.

Keep learned policies swappable. BotCortex need not train a foundation VLA; use policies as bounded executors underneath the task program. Pin weights and preprocessing separately from task code and adapter configuration.

9. Two teaching and adaptation lanes

Lane A — Simulation-first transfer

1. Bring up the robot adapter and calibrate the observed scene and frames.
2. Author object-centric intent and bind it to supported executors.
3. Rehearse in a versioned native simulator, including uncertainty in relevant parameters.
4. Review the candidate and perform supervised hardware trials.
5. Diagnose discrepancies, choose an intervention, and update the relevant artifact.
6. Save the verified skill and its operating envelope locally.

Play2Perfect, RialTo, TRANSIC, and calibration methods supply useful patterns, but have different prerequisites and costs. Do not compare their learning efficiency without accounting for pretraining, demonstrations, environment reconstruction, and human corrections. [1](#), [10–15](#)

Lane B — Direct hardware teaching

1. Collect demonstrations through a supported teleoperation or kinesthetic-teaching interface.
2. Record observations and **actually executed actions**, including human overrides and timestamps.
3. Train or adapt a bounded task executor, with a validated reward/outcome mechanism.
4. Use supervised interventions to gather recovery data.
5. Evaluate the executor on held-out initial states and store its capability envelope.

HIL-SERL and TRANSIC are relevant precedents for human-in-the-loop learning and residual correction. Their reported success should not be generalized to arbitrary robots, unseen tasks, or uninstrumented environments. [13](#), [14](#)

How the lanes meet

Both lanes use the same evidence schema, task contracts, artifact versioning, and outcome verifier. A simulation-first policy may acquire a residual from hardware corrections; a directly taught skill may later use simulation for diagnosis. Report the actual training path rather than assigning a misleading single label.

For the first paper, use a small, fixed repair-operator set. A bounded residual operator is optional if it would otherwise dominate schedule and confound the selector study. If omitted, narrow the final hypothesis and baseline matrix accordingly rather than claiming to evaluate it.

10. Experimental design

Robots and tasks

Use OpenArm and the Waveshare RoArm-M2-Pro as **two robot embodiments**. Use a single-manipulator task subset on OpenArm for the shared comparison; bimanual coordination is not a capability of the RoArm. In the RoArm's default clamp configuration, its three positioning joints do not independently set arbitrary tool orientation. Reconfiguring the fourth actuator as a wrist substitutes for clamp actuation; it does not add an independently powered gripper. [28](#)

Select shared object poses, approach directions, gripper clearances, loads, and tolerances that both setups can achieve in the pilot. Treat reach and stiffness differences as measurements to obtain, not established comparative results. Unsupported task/robot bindings are reported separately in coverage; they are not injected sim-to-real failures for the matched recovery comparison.

The RealSense is the **planned** shared RGB-D sensor. Identify the camera model, mount it, implement object detection/pose estimation or fiducial tracking, and validate timestamped observations and their uncertainty before using them. Record the camera-to-base transform for each arm separately and revalidate it whenever the camera, robot base, or workcell mounting changes. Factory calibration between camera sensors does not provide these robot/workcell transforms. [29](#)

Simulated embodiments

The two physical arms anchor the claim; simulated arms give it breadth. Use the vendored OpenArm MuJoCo model, a RoArm-M2-Pro model built from the vendor geometry, and three to five public arm models from MuJoCo Menagerie (candidates: Franka Panda, UR5e, xArm7, Kinova Gen3, SO-101), each bound through the same platform descriptor and conformance suite. [36](#) Menagerie models vary in dynamic fidelity; treat them as distinct kinematic/gripper embodiments for screening program, perception, and frame faults, and do not read simulated contact/physics transfer as a hardware result.

Each simulated embodiment declares a **supported-task matrix**: which task families it can bind, which it rejects, and why (reach, orientation control, gripper clearance). The RoArm's three positioning joints exclude tasks needing independent tool orientation. Honest rejection of an unsupported binding is itself tested. Within its supported set, every embodiment runs the same fault injections and verifier. The leave-one-embodiment-out protocol in the splits section below is run in simulation first, then on the two physical arms with the simulated bodies as source memory.

Start with four candidate task families: relational pick-and-place; pick/transport around bystanders; deliberate object pushing; and a contact-sensitive placement or insertion task supported by both setups. Use orientation-tolerant objects/fixtures where the RoArm cannot independently orient its

grasp. Insertion remains conditional on demonstrated approach feasibility and outcome observability. Deliberate pushing additionally requires extending the current no-unheld-object-motion guard with a task-specific allowed-effect set. Final selection follows the hardware pilot.

Failure benchmark

Fault group	Controlled examples	Important separation
Program / planning	Incorrect waypoint, missing prerequisite, wrong object relation.	Distinguish actual implementation errors from unreachable tasks.
Perception	Pose bias, occlusion, delayed observation, identity confusion.	Preserve measurement-validity labels.
Frame / TCP	Known transform or tool offset.	Keep injected configuration hidden from the selector.
Tracking / latency	Bounded delay, gain or velocity mismatch.	Verify actual response instead of assuming an injected setting took effect.
Contact / grasp	Object mass, surface friction, grasp offset, compliance changes.	Stay within the hardware operating envelope.
Compound / held-out	Unseen combinations and naturally occurring failures.	Allow ambiguous or multiple causes.

Validate controlled faults in simulation first. Hardware injections must be bounded, supervised, and appropriate to the mechanism. Unsafe or physically destructive faults are unnecessary to test the hypothesis.

Baselines under shared resources

1. Always repair the program/planner.
2. Always recalibrate or identify physical parameters.
3. Always use residual adaptation, **if that repair operator is implemented for all compared selectors.**
4. A Fail2Progress-inspired classification/repair strategy, with adaptations clearly stated.
5. A trace-only selector with no counterfactual replay or active measurements.
6. A simple rule-based selector over the same evidence and operators.
7. The proposed active selector with **no repair memory.**
8. The proposed active selector with **target-body memory only** (few-shot, k episodes).
9. The proposed active selector with **cross-body memory** (zero-shot on the target; the H1 treatment).
10. The proposed active selector with cross-body memory recorded as **joint-space traces plus free-text lessons** instead of contract violations (the H2 control).
11. An oracle-class selector for controlled faults, treated as a reference rather than a deployable method.

Arms 7 to 10 share one selector implementation and differ only in what memory retrieval is allowed to see.

Use the same candidate repair implementations and comparable observations, prompts/models, attempt limits, compute budgets, and available demonstrations wherever the comparison permits. Charge diagnostic actions against the relevant hardware/time budget. A published-method reproduction and a locally adapted baseline must be labelled differently.

Required ablations

- Remove contracts; retain traces.
- Remove counterfactual replay.
- Replace active measurement with passive evidence or random probes.
- Remove uncertainty-aware abstention.
- Remove capability/embodiment conditioning from retrieval (retrieve across bodies blindly).
- Remove version conditioning from retrieval.
- Restrict source memory to one embodiment at a time, to measure whether diversity of source bodies matters.

Use a staged design rather than running every ablation on every possible hardware condition immediately. Simulation can screen hypotheses; final claims need the relevant physical comparisons.

Splits and execution protocol

Leave-one-embodiment-out. For each embodiment E in the set, build the repair memory from all other embodiments' training episodes, then evaluate recovery on E's test faults. Run this over every simulated embodiment first. For the physical study, the two held-out targets are OpenArm and RoArm-M2-Pro; each is evaluated with source memory from the other physical arm plus the simulated bodies.

Freeze task/embodiment/fault combinations into training, validation, and test partitions. Include held-out combinations, not merely fresh random seeds of the same scenarios. Memory used at test time is a frozen snapshot; online updates during evaluation are disabled unless the experiment arm explicitly studies them. Randomize or counterbalance strategy order and record object resets, calibration changes, operator identity, and session/day effects.

For a concrete planning example on hardware, **2 physical embodiments × 4 tasks × 30 trials × 4 memory arms (baselines 7–10) = 960 evaluation episodes**, with fixed-strategy baselines and ablations screened in simulation and confirmed on a subset. This is an illustrative core comparison, not a finalized power calculation. The two OpenArm limbs are not counted as two distinct embodiments. Recovery attempts and diagnostic probes multiply the actual physical workload.

11. Metrics, statistics, and outcome verification

Primary and secondary measurements

Metric	Operational definition
Trials to verified recovery	Hardware task attempts from initial failure to independently verified success; report budget exhaustion.

Metric	Operational definition
Fixed-budget success	Fraction recovering within a prespecified attempt/time budget.
Human minutes	Setup, diagnosis, intervention, reset, and correction time, with categories visible.
Wrong-repair rate	Chosen intervention inconsistent with controlled fault labels or independent adjudication.
False-success rate	System reports success when the independent verifier or review finds failure.
Abstention and coverage	Frequency of refusing a diagnosis/execution and fraction of eligible cases actually handled.
Unchanged-program rate	Fraction transferred without modifying the task source hash; report adapter/calibration/weights separately.
Effort and runtime	Calibration, demonstrations, training compute, authoring, planning, simulation, hardware execution, and playback.
Transfer gain	Attempts-to-recovery (or fixed-budget success) on the held-out body with cross-body memory minus the same quantity with no memory; reported per failure class and per source/target pair.
Few-shot curve	Transfer gain as a function of k target-body episodes added to memory, $k \in \{0, 1, 5, 20\}$.
Negative-transfer rate	Fraction of episodes where cross-body memory led to a worse outcome than no memory; reported with the retrieved record that caused it.
Embodiment-distance sensitivity	Transfer gain against a declared capability-pack distance (joint count, actuation, gripper, sensing), to test whether degradation is predictable.

A repair can be practically effective without proving the original causal explanation. Report diagnosis accuracy and recovery performance separately. Also report failures to recover; do not calculate average recovery time only over successful episodes without clearly identifying that conditioning.

Verifier design

The existing runtime gate (`RobotSession.unverified`) checks execution *evidence*: an unrun skill, a refused run, a detected ejection, or a latched stop can never be reported as success. By its own docstring it does not grade the requested task, so a completed run plus `report(done=True)` does not establish that the object relationship was achieved. Keep that gate and add the research task verifier as a separate responsibility. In simulation the verifier reads hidden simulator ground truth through an interface the controller cannot see; the controller receives only the (possibly corrupted) observation stream. On hardware, use independent measurements for task success where possible: calibrated object pose, support/relation checks, grasp evidence, fixture state, and recorded video reviewed under a predefined rubric. Express uncertainty and observation failures. Evaluate reward classifiers and automatic verifiers against manually adjudicated samples, including borderline failures.

For perception/calibration faults, the verifier must not silently reuse the same corrupted estimates or transforms given to the controller. Keep an uncorrupted reference path for controlled injections, or use independent fixture/fiducial measurements and adjudicated video. Sharing one camera does not by

itself make outcome verification independent. The current `.ran` marker and a successful `run_skill` return are execution evidence, not ground-truth task-success labels.

Choose tolerance and dwell-time criteria before final runs. The current simulation guard's 15 mm threshold is an implementation setting for a specific transit check, not a universal benchmark success tolerance. Camera accuracy and task geometry determine valid physical thresholds.

Statistical reporting

- Pilot variance and failure frequency before selecting final sample sizes.
- Report confidence intervals, per-task/per-robot results, and aggregated results with their weighting rule.
- Account for repeated trials within robot, day, object, or task; use clustered analysis or an appropriate hierarchical model rather than pretending every attempt is independent.
- For budget-limited recovery, use a prespecified censored-time analysis or report fixed-budget success and restricted mean attempts; do not discard unrecovered episodes.
- Predefine the primary comparison and distinguish exploratory secondary analyses.

For perspective, **20 successes in 20 independent, identically distributed trials gives a one-sided exact 95% lower bound of approximately 86%: $0.05^{(1/20)} \approx 0.861$** . It does not establish 99% reliability, and dependence between repeated trials further limits the interpretation.

The existing BotCortex computation/playback timings demonstrate why latency categories matter. A simulator running faster or slower than real time should not silently change the reported robot execution speed.

12. Implementation roadmap and acceptance gates

The schedule below estimates the remaining study work, building on the existing runtime and resolved web fixes. Selecting the second robot and owning the camera are complete decisions; mounting, perception, adapters, and physical validation remain. The estimate assumes reliable hardware access and sufficient support. It is not a guarantee for one person working around hardware downtime. Physical access blocks calibration, measured controller behaviour, stopping tests, and hardware conformance; it does not block adapter interfaces, protocol handling against mocked or replayed serial feedback, unit and gripper conversions, conformance-test definitions, or perception software against recorded and synthetic observations. Those run ahead of the rig.

Phase	Estimate	Work and acceptance gate
0. Scope and hardware	1–2 weeks	With RoArm-M2-Pro selected and RealSense owned, validate the controller link and default clamp mode; identify/mount the camera, establish transforms and tracking feasibility; freeze initial tasks, fault taxonomy, evidence schema, and pilot protocol.
1. Adapters and perception	3–4 weeks	Two real adapters pass conformance; frames/timestamps validated; bounded motion and stopping tested; outcome sensing works.

Phase	Estimate	Work and acceptance gate
2. Contracts and benchmark	3 weeks	Extend existing rehearsal, route guards, logs, and memory into versioned contracts/replay packets, controlled faults, an independent verifier, and simple baselines.
2b. Multi-embodiment simulation benchmark	3–4 weeks	Bind a RoArm model and three to five Menagerie arms through the platform descriptor; run every task family and fault injection on every body; run leave-one-embodiment-out in simulation. Gate: transfer gain is measurable in simulation for at least program and perception faults, or the hypothesis is narrowed before hardware.
3. Diagnosis and selection	3–4 weeks	Passive and active selectors share repair operators; ambiguity/abstention supported; simulation pilot shows measurable value.
4. Hardware evaluation	4–6 weeks	Freeze splits/configuration and the memory snapshots; run the four memory arms on both physical targets; confirm fixed-strategy baselines and justified ablations on a subset; include unsuccessful repairs, negative transfer, and natural failures.
5. Paper and artifacts	2 weeks	Statistical analysis, figures, claim audit, reproducibility package, videos, and submission formatting.

The phases sum to approximately **19–25 weeks** if largely sequential. Some analysis and writing can overlap; procurement, debugging, and experimental resets can extend the calendar.

Concrete engineering work packages

1. **Capability schema:** remove global platform assumptions and define versioned units, frames, modes, limits, sensors, and gripper semantics. Gate: instantiate both platforms without shared-state contamination.
2. **Adapter conformance:** test command/state mapping, timestamps, bounded control, interruption, and unavailable capabilities. Gate: the same suite passes both adapters with explicit capability-specific expectations.
3. **Perception bridge:** replace privileged simulation lookups with a shared observation contract. Gate: identical task code can consume simulated and measured estimates, with uncertainty preserved.
4. **Contract engine:** generalize the existing collision, grasp, and bystander checks to task-specific preconditions, effects, invariants, and unknown verdicts. Gate: intentional pushing is allowed while bystander damage is rejected.
5. **Evidence recorder:** extend current logs and run-bound records with synchronized hardware actions, observations, versions, and overrides. Gate: a disputed primitive can be reconstructed and reviewed without agent narration.
6. **Replay and fault harness:** expose versioned replay packets over existing state restoration, inject bounded faults, and compare hypotheses. Gate: preserve the already-fixed diagnostic-query purity and validate replay within stated numerical tolerances.
7. **Repair operators:** implement bounded, versioned program and calibration/physics repairs first. Gate: each operator has a measurable cost and independently verified outcome.

8. **Selector and active probes:** establish fixed/rule-based baselines before a learned selector. Gate: active probes add value after accounting for their cost.
9. **Embodiment-aware memory:** extend existing account-scoped persistence with capability-pack identity, version conditioning, switchable cross-body retrieval, and experimental split boundaries. Gate: held-out embodiments cannot leak through the retrieval index; the same query returns different result sets under the four memory arms; browser reload/ownership fixes stay covered.
10. **Embodiment catalogue:** RoArm model plus Menagerie arms as `platforms/` directories with descriptors and conformance results. Gate: adding an arm is a directory plus a supported-task matrix; every arm runs its supported task families in headless simulation and rejects the rest with a stated reason.
11. **Evaluation runner and paper artifact:** freeze configurations and export auditable episodes/statistics. Gate: rerunning analysis reproduces tables and includes failed/budget-exhausted cases.

Go/no-go decisions

- **After phase 1:** if the RealSense does not give reliable object-state verification at the task tolerances, or the RoArm-M2-Pro cannot execute the shared tasks, resolve that dependency before claiming a two-robot study.
- **After phase 2:** if controlled failure classes cannot be distinguished even with extra sensing, narrow the taxonomy or study uncertainty rather than forcing labels.
- **After phase 2b:** if cross-body memory shows no transfer gain in simulation even for program and perception faults, first rule out weak retrieval, too little source memory, unsuitable faults, and implementation defects. Only after that diagnosis does the thesis fall back to H3 (active selection) with the benchmark still released. A negative H1 pilot is a decision gate, not a finding, and it does not establish H3. Do not carry an undiagnosed H1 onto hardware.
- **After phase 3:** if simple rules match the proposed selector, improve the mechanism before expanding physical trial count.
- **Before submission:** if only simulation is complete, present a simulation study with that scope; do not describe promised hardware experiments as results.

13. Resources, prerequisites, and main risks

Hardware and instrumentation

The second arm is the Waveshare RoArm-M2-Pro; the shared sensor is an Intel RealSense depth camera that is owned but not yet mounted. The camera model and installed firmware still need recording. Camera-based perception and camera-to-base fault experiments await a calibrated observation pipeline. Some TCP/frame checks can instead use an independently measured fixture or mechanical reference; absence of a camera does not make every calibration experiment impossible.

Neither arm has a dedicated external force/torque sensor in the recorded setup. This does not mean all load feedback is absent: the RoArm documentation advertises servo feedback, and JSON T=105 exposes joint angles and load indicators. Those indicators and advertised torque-limit controls are not calibrated contact-force measurements in newtons. Validate available telemetry on the actual firmware, and restrict force/contact claims to what the instrumentation supports. A contact-sensitive insertion task requires both feasible motion and independent outcome verification. [28](#)

Bring-up requires calibrated camera-to-base and tool transforms for each arm, explicit degree/radian and metre/millimetre mappings for the chosen controller interface, validated gripper conventions, suitable fixtures, and synchronized recordings across the host, camera, and RoArm controller. Reuse the existing OpenArm teach/replay helpers where appropriate, while validating their integration into the BotCortex execution/evidence path. Real motion remains supervised and explicitly authorized, with independent stop facilities for both setups. No hardware connection or motion was performed for this review.

Compute and data

Use existing primitives and planners for early experiments, then add small task policies only where needed. Run bulk simulation headlessly. Budget separately for parameter search, policy adaptation, authoring calls, data storage, and human annotation. Keep pretrained-model and demonstration costs in comparisons even when amortized.

Play2Perfect's week-long pretraining and per-task finetuning illustrate a different compute commitment from this proposed selector study. It is a useful later executor, not a prerequisite for the first paper. [1](#)

Risks and mitigation

Risk	Practical response
Observationally equivalent causes	Add targeted measurements and calibrated abstention; avoid forced single labels.
Simulator explains the wrong mechanism	Use uncertainty ensembles and validate predicted intervention effects on hardware.
Weak or circular verifier	Independently adjudicate representative successes, failures, and ambiguous cases.
Repair baseline is artificially weak	Share operators, observations, and budgets; include simple rules and strong adapted methods.
Cross-robot differences dominate	Match task semantics, report capabilities, and separate calibration from program changes.
Memory contaminates held-out evaluation	Freeze indexes/splits and audit artifact provenance.
Trial workload overwhelms schedule	Pilot first; prioritize primary comparisons; record resets and diagnostic attempts.

Risk	Practical response
Apparent novelty disappears	Update the closest-work review and narrow the contribution before submission.

14. Publication strategy

Choose the venue based on demonstrated evidence. A strong learning/decision method with real transfer results can fit **CoRL** or **RSS**. A rigorous robotic method and integrated system can fit **ICRA** or **IROS**. A workshop is useful for early feedback, but is not equivalent to acceptance of a full conference paper.

Deadline snapshot — checked 11–12 September 2026

Venue	Verified planning information	Implication
ICRA 2027	Official CFP lists 15 September 2026 as the paper deadline. 23	Too soon for the proposed new hardware study.
CoRL 2026	Submission window has passed; the author instructions describe double-blind review and an initial 8-page paper. 24 , 25	Use for formatting expectations, not as an available submission window.
RSS 2027	Announced 6–11 July 2027 in Athens ; a 2027 paper deadline was not verified. 26	Track the official CFP; do not infer a date from a prior year.
IROS 2027	IEEE RAS event listing gives 1 March 2027 for paper submission. 27	Potential planning target; confirm against the final conference CFP.

Dates are a historical snapshot and must be rechecked before scheduling a submission. No CoRL 2027 deadline is asserted here.

Submission package

The paper should contain a precise problem definition, clear separation from Fail2Progress/COMPASS/ASID, LabRobFail, cross-embodiment policy transfer, and program-repair work; a fully specified selector and memory representation; strong equal-budget baselines including the no-memory and target-only arms; leave-one-embodiment-out results in simulation and on both physical arms; ablations; uncertainty analysis; and honest negative-transfer cases. Release enough configurations, code, traces, and analysis for another lab to understand what changed between attempts.

Keep the method contribution separate from product positioning. “Teach your robot by typing” explains the product; “a failure repaired on one robot body makes recovery cheaper on another” is a testable paper claim.

The recommended target is a credible **2027 submission**, with venue choice following the actual result rather than rushing to an imminent deadline. Acceptance cannot be predicted from a proposal.

15. Claim ledger and final recommendation

Claim	Current status	Evidence needed to strengthen it
The captured blue-block simulation failure was fixed.	Supported for the reproduced task and tested variants.	Broader task/perturbation benchmark for generalization.
Native and browser paths share useful runtime behavior.	Supported by the recorded regression checks.	Expanded semantic conformance and performance characterization.
Rehearsal, route guards, browser persistence, and run-routing fixes exist.	Implemented; previous verification is recorded.	Extend them for hardware evidence, task contracts, and experiment isolation.
The second robot and camera have been chosen.	RoArm-M2-Pro selected; RealSense owned, mounting pending.	Validate controller/firmware, camera model, transforms, and pose pipeline.
BotCortex is already a hardware-portable SDK.	Not established; four simulated bodies now run through one descriptor, no real adapter yet.	Two implemented real adapters, sensing, binding, and conformance evidence.
A completed skill run proves the requested task succeeded.	Not established by the current .ran marker or return status.	Independent task-effect verification, especially under perception/calibration faults.
The same task program can transfer across both selected arms.	Research objective.	Frozen task hashes plus separately reported adapter/calibration changes.
Repair knowledge transfers across robot bodies (H1).	Supported in simulation only: 4.61 → 2.00 attempts on held-out bodies, 5/28 negative-transfer cases (15 Sep 2026 pilot).	Physical results on both arms; more task families and faults; a targeted-measurement step to separate bias from stale pose.
The transfer depends on the contract representation (H2).	Weakly supported in simulation: contract 2.00 vs trace 2.32, right family 26/28 vs 21/28.	Faults whose traces and contract violations disagree; a trace arm with agent-written lessons.
Active diagnosis reduces recovery effort (H3).	Secondary hypothesis.	Equal-budget physical comparisons and statistical analysis.
No prior work transfers repair knowledge across embodiments.	Supported by a targeted discovery pass on 12 September 2026; not by citation tracing.	Full related-work tracing before submission; the claim narrows if a match appears.
Replay can always identify the root cause.	Not a defensible claim.	Use bounded identifiability statements and unresolved cases.
Browser simulation or skill memory is new.	Contradicted by close prior work.	Novelty must come from cross-body transfer and its evidence.

Claim	Current status	Evidence needed to strengthen it
A small perfect trial set proves near-perfect reliability.	Unsupported.	Appropriate uncertainty estimates and larger, diverse trials.
A major-conference paper is guaranteed.	Not knowable.	Strong completed research and peer review.

Recommended next implementation milestone

Do not complete the workstreams one after another. Build **one thin end-to-end slice** through runtime, method, and evaluation, in simulation, before widening any of them:

1. One shared task (relational pick-and-place) bound on a few simulated embodiments: OpenArm, a RoArm model, and two or three Menagerie arms, each with a declared supported-task matrix.
2. An independent verifier reading hidden simulator truth, plus controlled fault injection for program and perception faults, with the ground truth hidden from the selector.
3. Recorded source failures, the repair applied, and the verified outcome, stored with embodiment identity and artifact versions.
4. One fixed selector (rule-based is enough) run under the four memory arms: none, target-only, cross-body, cross-body-as-traces.
5. A frozen held-out-body evaluation that also excludes the target's own simulated variants from the source memory.

That slice yields the first H1 signal with no hardware and no active-probe system, and it forces the verifier and split design to exist *before* benchmark memory is collected, so the episodes gathered can support the eventual claim. In parallel, the hardware track establishes **one objectively verified, orientation-feasible task on OpenArm and RoArm-M2-Pro** with versioned adapters and calibration, using the RealSense once mounted or an independently measured fixture until then.

Those two milestones together provide an honest foundation for the SDK, the benchmark, and the paper. Dexterous pretraining, a large skill marketplace, and fleet-scale transfer build on them later without enlarging the first research question.

16. Sources and research notes

References are linked to primary papers, repositories, or official conference pages. “Core review” means the relevant main-text methods/results and selected supporting sections were examined during the research, not that every appendix or citation was independently reproduced. Background entries support context and further reading; they are not evidence for unreported BotCortex performance.

[1] Play2Perfect

Play2Perfect: What Matters in Dexterous Play Pretraining for Precise Assembly? arXiv 2606.26428, reviewed **v3 (8 September 2026)**. Core review, including training setup, hardware results, limitations, and selected appendices.

- Paper: <https://arxiv.org/abs/2606.26428v3>
- Full text: <https://arxiv.org/html/2606.26428v3>
- Project: <https://play2perfect.github.io/>
- MIT-licensed code: <https://github.com/kushal2000/play2perfect>
- Deployment guide:
<https://github.com/kushal2000/play2perfect/blob/main/docs/deployment.md>

[2] DexGPT

Hu-xiao-max/dexgpt, repository snapshot **03ba8a26eaef9dfd272ac1a22c0fd06ccb9b2e5b**.
Repository/code/report review; no top-level license identified at the snapshot.

- Pinned repository: <https://github.com/Hu-xiao-max/dexgpt/tree/03ba8a26eaef9dfd272ac1a22c0fd06ccb9b2e5b>
- Inspect `dexgpt/motion/retarget.py`, `dexgpt/simulation/rollout.py`, and `outputs/physics_report.json` in that snapshot.

[3] ASPIRE

ASPIRE, arXiv **2607.00272**. Core review of program search, trace feedback, reusable skills, and transfer examples. A close comparison for program repair and skill-library claims.

<https://arxiv.org/abs/2607.00272>

[4] CaP-X

CaP-X, arXiv **2603.22435**, reviewed **v2**. Core review of coding-agent evaluation and the perception/control harness.

<https://arxiv.org/abs/2603.22435v2>

[5] Nautilus

Nautilus, arXiv **2605.11665**. Core interface/architecture review. Relevant to typed contracts, isolated execution, and the distinction between portable interfaces and successful policy transfer.

<https://arxiv.org/abs/2605.11665>

[6] Fail2Progress

Yixuan Huang, Novella Alvina, Mohanraj Devendran Shanthi, and Tucker Hermans. *Fail2Progress: Learning from Real-World Robot Failures with Stein Variational Inference*. arXiv **2509.01746v1**, CoRL 2025. Core failure-reconstruction and limitation review; bibliographic metadata rechecked for this dossier.

<https://arxiv.org/abs/2509.01746v1>

[7] Counterexample-guided abstraction repair

Counterexample-guided abstraction repair, arXiv **2105.06537**. Targeted closest-work review of counterexample-based symbolic/geometric repair.

<https://arxiv.org/abs/2105.06537>

[8] COMPASS / What Went Wrong?

Peide Huang and colleagues. *What Went Wrong? Closing the Sim-to-Real Gap via Differentiable Causal Discovery*. CoRL 2023, PMLR **229:734–760**. Targeted causal-calibration review; official proceedings metadata and abstract rechecked for this dossier.

- Proceedings: <https://proceedings.mlr.press/v229/huang23c.html>
- Paper: <https://proceedings.mlr.press/v229/huang23c/huang23c.pdf>

[9] RAPT

RAPT: Model-Predictive Out-of-Distribution Detection and Failure Diagnosis for Sim-to-Real Humanoid Deployment. arXiv **2602.01515**. Targeted review of prediction, localization, and diagnosis.

<https://arxiv.org/abs/2602.01515>

[10] ASID

ASID: Active Exploration for System Identification in Robotic Manipulation. arXiv **2404.12308**. Core review of active physical identification.

<https://arxiv.org/abs/2404.12308>

[11] Phys2Real

Phys2Real: Fusing VLM Priors with Interactive Online Adaptation for Uncertainty-Aware Sim-to-Real Manipulation. arXiv **2510.11689**, reviewed **v2**. Core review of physical priors and interactive adaptation.

<https://arxiv.org/abs/2510.11689v2>

[12] Neural Fidelity Calibration

Neural Fidelity Calibration for Informative Sim-to-Real Adaptation. arXiv **2504.08604**. Targeted review of informative simulator adaptation.

<https://arxiv.org/abs/2504.08604>

[13] TRANSIC

TRANSIC: Sim-to-Real Policy Transfer by Learning from Online Correction. arXiv **2405.10315**, reviewed **v3**. Core review of base-policy transfer, gated residuals, and human corrections.

<https://arxiv.org/abs/2405.10315v3>

[14] HIL-SERL

HIL-SERL, arXiv **2410.21845**. Core paper/project review of direct hardware learning, demonstrations, interventions, and reward classifiers.

- Paper record: <https://arxiv.org/abs/2410.21845>
- Project: <https://hil-serl.github.io/>
- Project paper: <https://hil-serl.github.io/static/hil-serl-paper.pdf>

[15] RialTo

RialTo: A Real-to-Sim-to-Real Approach for Robust Manipulation. arXiv **2403.03949**. Abstract/project review for reconstructed-environment robustification; no experiment reproduced here.

- Paper: <https://arxiv.org/abs/2403.03949>
- Project: <https://real-to-sim-to-real.github.io/RialTo/>

[16] CEI

CEI: A Unified Interface for Cross-Embodiment Visuomotor Policy Learning in 3D Space. arXiv **2601.09163**. Core representation/trajectory-alignment review, including functional representations and directional Chamfer distance.

<https://arxiv.org/abs/2601.09163>

[17] UHAS

Cross-Embodiment Robot Manipulation via a Unified Hand Action Space. arXiv **2607.03570**. Core transfer/limitations review. The reviewed real LEAP setup still required identified gains/velocity limits and matched **20 Hz** control because of serial bandwidth; shared action space did not remove hardware calibration.

<https://arxiv.org/abs/2607.03570>

[18] ReKep

ReKep: Spatio-Temporal Reasoning of Relational Keypoint Constraints for Robotic Manipulation. arXiv **2409.01652**. Targeted review of relational constraints and closed-loop optimization.

<https://arxiv.org/abs/2409.01652>

[19] Grounding Sim-to-Real Generalization

Grounding Sim-to-Real Generalization, arXiv **2603.22876**. Supporting transfer-evaluation source; full text was collected, with a narrower review than the closest-work papers.

<https://arxiv.org/abs/2603.22876>

[20] DrEureka

DrEureka, arXiv **2406.01967**. Background discovery/abstract-project review for language-assisted reward and domain-randomization design.

<https://arxiv.org/abs/2406.01967>

[21] Re³Sim

Re³Sim, arXiv **2502.08645**. Background discovery/abstract review for real-to-sim reconstruction and manipulation data generation.

<https://arxiv.org/abs/2502.08645>

[22] RobolInspector

RobolInspector, arXiv **2508.21378**. Supporting source for failure-diagnostic feedback; also cited in the existing company/runtime research. Its reported gains are not reproduced in this dossier.

<https://arxiv.org/abs/2508.21378>

[23] ICRA 2027 official call for papers

Official submission deadline source, checked during the 11–12 September 2026 research snapshot.

<https://2027.ieee-icra.org/contribute/call-for-icra-2027-papers-now-accepting-submissions/>

[24] CoRL author instructions

Official author guidance consulted for review format and initial paper length; future-year rules may differ.

<https://www.corl.org/contributions/instruction-for-authors>

[25] CoRL call for papers

Official conference contribution information for the 2026 window. Recheck the site for the next cycle.

<https://www.corl.org/contributions/call-for-papers>

[26] Robotics: Science and Systems

Official conference site announcing RSS 2027 in Athens, 6–11 July. A 2027 paper deadline was not verified in this review.

<https://roboticsconference.org/>

[27] IROS 2027 IEEE RAS event listing

Official society event listing consulted for the 1 March 2027 paper-submission date; confirm against the final conference CFP.

<https://www.ieee-ras.org/event/2027-ieee-rsj-international-conference-on-intelligent-robots-and-systems-iros-70525/>

[28] Waveshare RoArm-M2 family: hardware, tooling, and command interface

Official product, wiki, and SDK documentation verified through Aside search during this revision. The shared product listing and firmware repository explicitly name M2-S and M2-Pro; the control/EoAT wiki pages retain M2-S titles. These document the default clamp versus wrist configuration, JSON radian positions and Cartesian millimetres, step-based speed/acceleration fields, load feedback, and the SDK's separate gripper convention. They do not establish measured performance or the installed firmware configuration of the owner's arm.

- Shared product listing: <https://www.waveshare.com/roarm-m2-s.htm>
- Joint/Cartesian control and feedback: https://www.waveshare.com/wiki/RoArm-M2-S_Robotic_Arm_Control
- Clamp/wrist configuration: https://www.waveshare.com/wiki/RoArm-M2-S_EoAT_Setting
- Shared firmware repository: https://github.com/waveshareteam/roarm_m2
- M2 Python SDK conventions:
https://github.com/waveshareteam/waveshare_roarm_sdk/blob/main/doc/roarm_m2_en.md

The movement-control wiki has inconsistent unit wording in its T=121 heading and in the Cartesian t description. Use the clearly documented T=101/102 angle convention as a starting point, and validate every used command/feedback field against the pinned firmware. Do not copy unverified numeric limits or treat every JSON field as having the same unit.

[29] RealSense SDK: acquisition and calibration scope

Official RealSense SDK documentation checked through Aside. The SDK provides depth/color streaming, sensor calibration information, aligned streams/point clouds, and recording/playback. For the proposed workcell, object-pose estimation/tracking and camera-to-robot calibration remain

application-level integration and validation work. No particular owned camera model, frame rate, pose accuracy, or completed installation is inferred from these general docs.

- SDK repository: <https://github.com/realsenseai/librealsense>
- SDK overview: <https://www.realsenseai.com/news-insights/intel-realsense-sdk-2-0>

[30] Scaling Cross-Environment Failure Reasoning Data for Vision-Language Robotic Manipulation

arXiv **2512.01946**. Abstract-level discovery, 12 September 2026. Closest framing for failure knowledge across environments; single embodiment, model-training focus.

<https://arxiv.org/abs/2512.01946>

[31] LabRobFail

LabRobFail: A Benchmark for Robotic Failure Analysis in Chemical Self-driving Laboratory. arXiv **2607.23704**. Abstract-level discovery, 12 September 2026. Simulated fault injection with five failure categories, eleven fine-grained types, and 20,000+ trajectories on one lab robot. The closest benchmark; ours must add physical embodiments and repair-outcome labels.

<https://arxiv.org/abs/2607.23704>

[32] Causal Agent Replay

Causal Agent Replay: Counterfactual Attribution for LLM-Agent Failures. arXiv **2606.08275**. Abstract-level discovery. Software-agent counterfactual attribution via do-operations on a recorded run.

<https://arxiv.org/abs/2606.08275>

[33] AgentDebugX

AgentDebugX: An Open-Source Toolkit for Failure Observability, Attribution, and Recovery in LLM Agents. arXiv **2607.18754**. Abstract-level discovery. Detect / attribute / recover / rerun loop for software agents.

<https://arxiv.org/abs/2607.18754>

[34] Repair or Resample?

Repair or Resample? Rethinking Failure Debugging in LLM Multi-Agent Systems. arXiv **2608.25920**. Abstract-level discovery. Replay from intervention anchors in recorded multi-agent traces.

<https://arxiv.org/abs/2608.25920>

[35] Data Analogies Enable Efficient Cross-Embodiment Transfer

arXiv **2603.06450**. Abstract-level discovery. Paired, aligned demonstrations across embodiments drive policy transfer under morphology shift; the closest analogue for aligned cross-body data.

<https://arxiv.org/abs/2603.06450>

[36] MuJoCo Menagerie

Google DeepMind's curated collection of MuJoCo robot models, including several manipulator arms. Repository consulted for the candidate simulated embodiments; per-model fidelity and licences must be checked before use.

https://github.com/google-deepmind/mujoco_menagerie

[37] Diagnose, Correct, and Learn from Manipulation Failures via Visual Symbols

arXiv **2512.02787**. Abstract-level discovery. Symbolic failure diagnosis and correction on a single robot.

<https://arxiv.org/abs/2512.02787>

Local evidence and reproducibility notes

- `docs/SIMULATION_FAILURE_FIX.md` : captured failure, native/WASM outcomes, timings, verification history, source commit, and artifact provenance.
- `docs/ARCHITECTURE_AUDIT.md` and the **Resolution** section of `docs/CLAUDE_REVIEW.md` : distinguish resolved R1–R9 findings from remaining runtime/API research requirements. These are implementation records, not hardware study results.
- Current source check: web baseline `37b5452` ; runtime baseline `1f92197` . `robot.py / cli.py` establish the missing package hardware path; `sim.py / wasm.py` establish existing rehearsal restoration; `session.py / skills.py` distinguish run completion from task success.
- Thor/OpenArm lab runbook: records separate LeRobot teach/replay helpers and a joint-space sim bridge. This review read the local runbook without connecting to the rig; integration or fresh hardware performance is not inferred.
- `public/botcortex/MANIFEST.json` : shipped runtime/wheel pin. Recorded wheel SHA-256: `c657c6e094e7db4970068b7ff2ee3eb71c189335855961a474bf084054e1387d` .
- Sibling `botcortex-runtime` source: `platform.py` , `config.py` , `jointmap.py` , `robot.py` , `kinematics.py` , `scene.py` , `skills.py` , `session.py` , `sim.py` , and `wasm.py` establish the inspected implementation boundaries.
- `.firecrawl/sim2real/` : original ignored literature cache. The added hardware checks used Aside search; sources [28]–[29] are the portable public record of those findings.

End of dossier. Research and deadline statements reflect the 12 September 2026 snapshot. Proposed architecture, timelines, and experiments remain subject to the stated implementation and evidence gates.